

Diseño de la Interfaz de una App usando IA

AulaMatch — Sistema de Gestión de Espacios Académicos

Materia: Herramientas de IA aplicadas al Diseño

Fecha: Junio 2025

Origen del proyecto: La idea de AulaMatch surge de una problemática real y cotidiana planteada por la Directora de Aplicaciones de la Universidad Nacional de San Martín (UNSAM), quien identificó el proceso de asignación de aulas como uno de los cuellos de botella administrativos más recurrentes al inicio de cada cuatrimestre. Los equipos de administración gestionaban todo manualmente en planillas Excel.

SECCIÓN 1

Identificación del Problema y Propuesta de Solución

El problema

Las universidades organizan su oferta académica a través de Unidades Académicas (UA), cada una de las cuales dicta sus propias materias. A su vez, cada materia puede tener múltiples comisiones simultáneas con diferentes docentes, horarios y modalidades (presencial, virtual o híbrida), lo que multiplica exponencialmente la cantidad de espacios físicos a coordinar. Este proceso de asignación de aulas se renueva íntegramente cada cuatrimestre, lo que significa que el volumen de trabajo administrativo se repite desde cero de forma periódica. Gestionado de manera manual o con herramientas básicas, el proceso genera conflictos de horario, subutilización de espacios y una marcada dificultad para responder a cambios imprevistos —como materias que se incorporan a mitad de cuatrimestre o emergencias edilicias— consumiendo tiempo administrativo considerable y siendo propenso a errores humanos con impacto directo en la operación académica.

Usuarios y contexto de uso

AulaMatch está dirigido al personal administrativo de cada Unidad Académica y a los coordinadores de espacios físicos de la institución. Estos usuarios interactúan con el sistema en dos momentos clave: al inicio de cada cuatrimestre, cuando deben planificar y asignar aulas en forma masiva para todas las comisiones activas —considerando cupos, modalidades, tipos de aula y edificios de preferencia de cada UA—, y de forma puntual durante el cuatrimestre, cuando surgen situaciones imprevistas que obligan a revisar o reasignar espacios de manera urgente. Se trata de usuarios con perfil operativo y de coordinación, que necesitan una herramienta ágil, confiable y fácil de usar sin necesidad de formación técnica especializada.

Funcionalidades clave

- 1. Asignación automática de aulas:** Al inicio de cada cuatrimestre, el sistema propone asignaciones optimizadas para todas las comisiones considerando cupo, tipo de clase requerido, franja horaria y edificio preferido de cada UA. Elimina horas de trabajo manual y reduce el margen de error en la planificación masiva.
- 2. Asignación manual asistida:** Cuando el coordinador necesita intervenir directamente, el sistema filtra y muestra solo las aulas disponibles que cumplen con las características requeridas por la comisión, permitiendo decisiones informadas.
- 3. Gestión de excepciones en tiempo real:** Ante la incorporación de una nueva materia o comisión a mitad de cuatrimestre, el sistema permite una asignación rápida mostrando disponibilidad actualizada al instante.
- 4. Detección y alerta de conflictos:** El sistema identifica automáticamente situaciones problemáticas —superposición de horarios, cupo excedido— y notifica al usuario antes de que el conflicto tenga impacto real.

SECCIÓN 2

Prompt para Generación Visual en Midjourney

El siguiente prompt fue elaborado para generar una imagen conceptual de la interfaz y el estilo visual general de AulaMatch, siguiendo lineamientos de diseño flat, paleta institucional y estructura de dashboard universitario.

UI/UX design mockup of a modern university classroom management web application called "AulaMatch". Clean, professional dashboard screen. Dark sidebar navigation on the left with menu items: Dashboard, Asignaciones, Comisiones, Aulas, Edificios, Carreras, Reportes. Main content area shows a filterable data table with a white/light grey background. At the top of the main area: page title "Asignaciones – 2do Cuatrimestre 2025" and a row of filter controls: dropdown for "Unidad Académica", dropdown for "Carrera", dropdown for "Edificio", dropdown for "Turno" (Mañana / Tarde / Noche), dropdown for "Modalidad" (Presencial / Virtual / Híbrida), and a blue "Asignar automáticamente" action button on the right. Below the filters: a dense data table with columns: Comisión | Materia | Carreras | UA | Docente | Cupo | Aula asignada | Edificio | Horario | Modalidad | Estado. The "Carreras" column shows compact multi-value chips/tags. Each row has a colored status badge: green "Asignada", yellow "Pendiente", red "Conflicto". Column headers have sort arrows. Table has alternating row shading. Top right: user avatar, notification bell with red badge counter. Color palette: deep navy blue sidebar (#1E2A45), white main content, blue accent buttons (#2563EB), clean sans-serif typography. Pixel-perfect flat UI design. No gradients. 16:9, high resolution.

El resultado generado por Midjourney sirvió como referencia conceptual visual antes de trasladar el diseño a Figma AI para el wireframe funcional.

Imagen generada por Midjourney — AulaMatch Dashboard

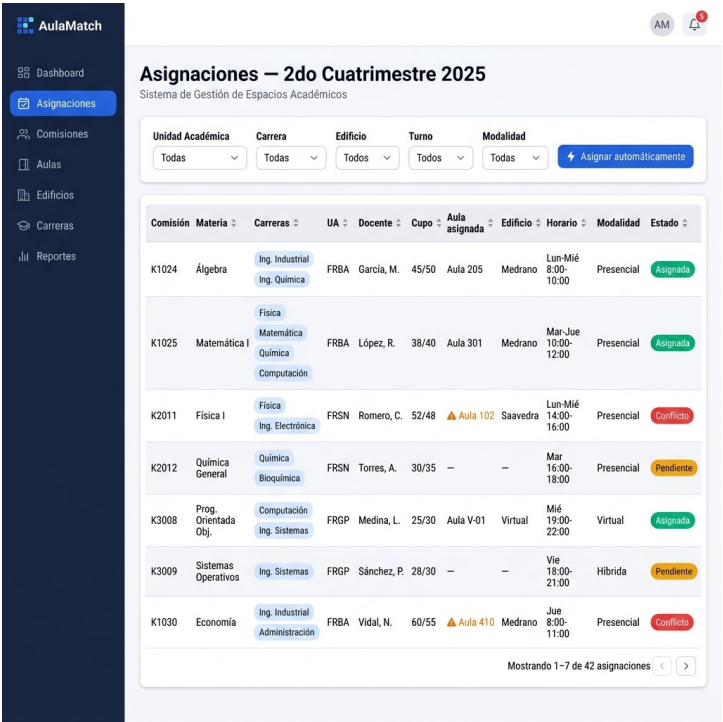


Figura 0 — Resultado de Midjourney: mockup conceptual del dashboard de AulaMatch (sidebar navy, tabla densa, chips de carrera, badges semánticos).

SECCIÓN 3

Wireframe Funcional — Figma AI

Las siguientes capturas corresponden al wireframe funcional de AulaMatch diseñado en Figma AI. Se presentan cuatro pantallas que cubren los flujos principales del sistema: inicio de sesión, asignaciones (pantalla principal), panel de conflictos y perfil/configuración.

Pantalla 1 — Inicio de Sesión

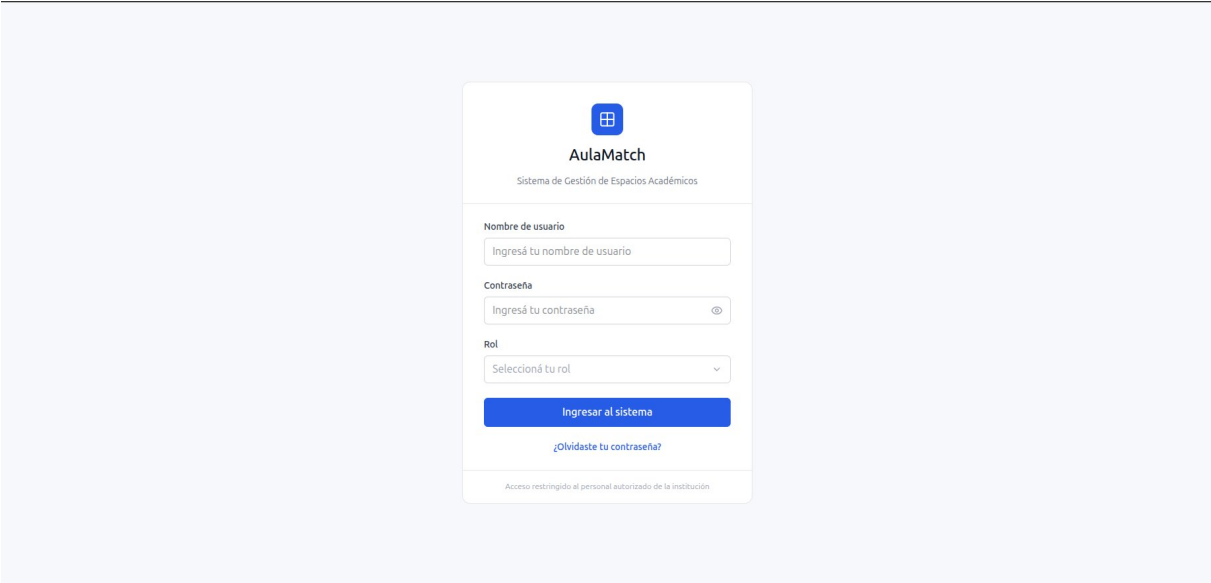


Figura 1 — Pantalla de bienvenida/authenticación. Permite al personal administrativo ingresar con usuario, contraseña y selección de rol institucional.

Pantalla 2 — Asignaciones (Pantalla Principal)

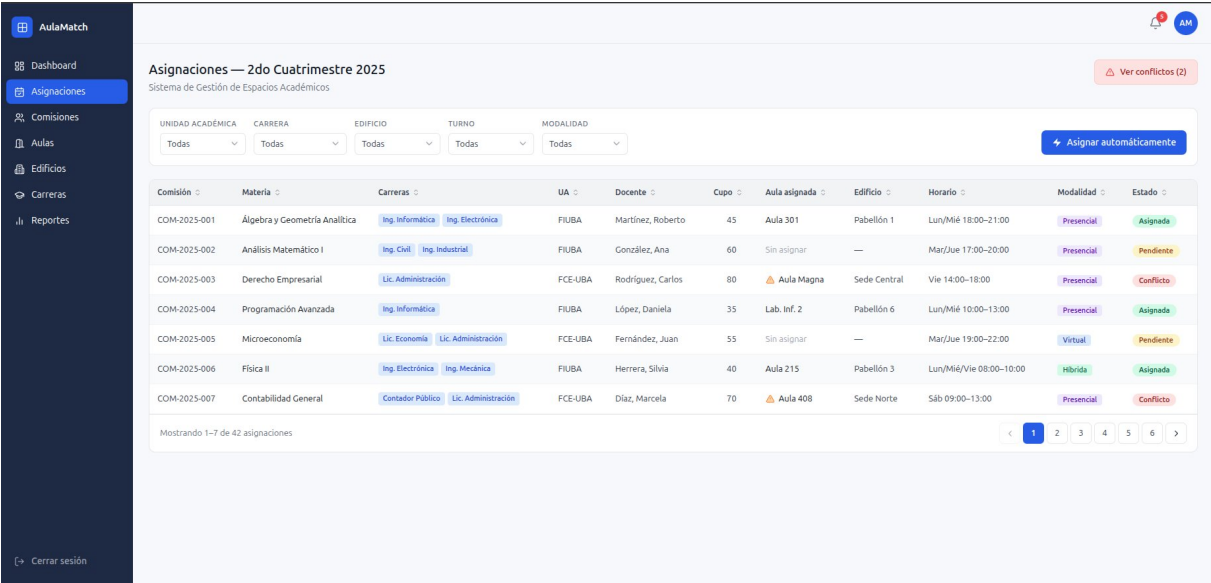


Figura 2 — Vista central de gestión de asignaciones. Filtros por UA, Carrera, Edificio, Turno y Modalidad. Tabla con 11 columnas y badges de estado semánticos.

Pantalla 3 — Panel de Conflictos

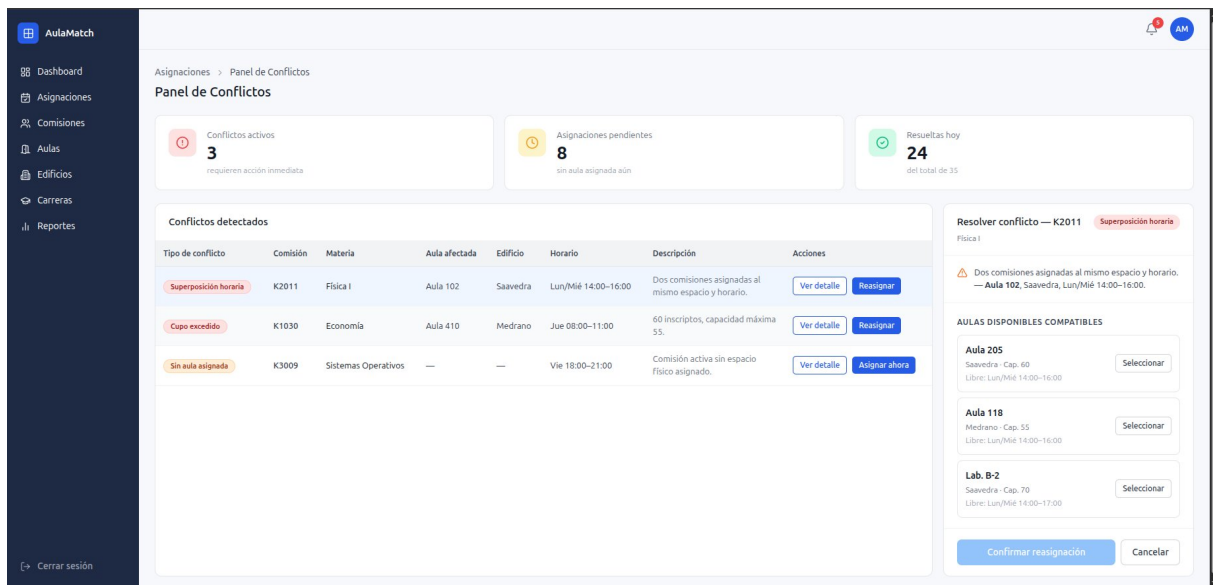


Figura 3 — Detección y resolución de conflictos en tiempo real. Cards métricas, tabla de conflictos y panel lateral de resolución asistida.

Pantalla 4 — Perfil y Configuración (parte 1)

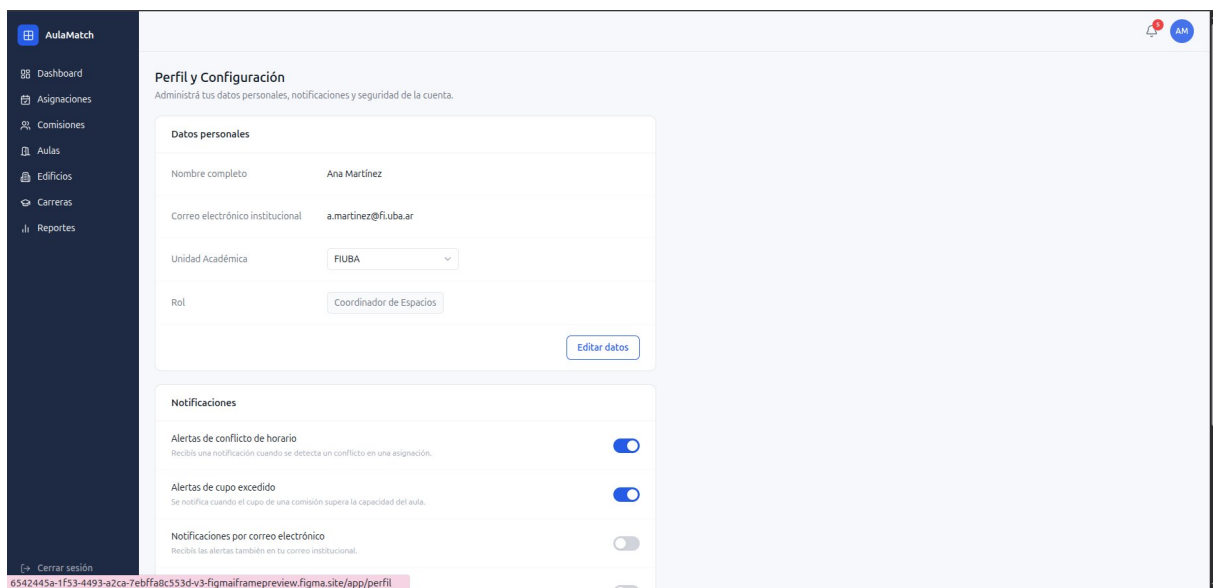


Figura 4a — Datos personales e institucionales del usuario, campos editables y selector de Unidad Académica.

Pantalla 4 — Perfil y Configuración (parte 2)

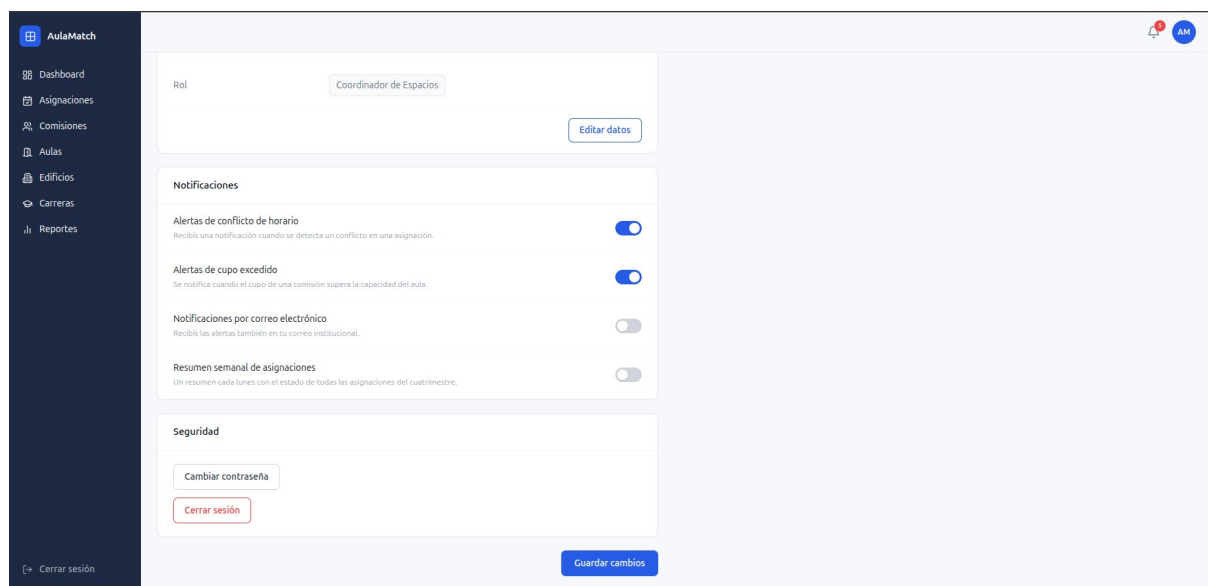


Figura 4b — Configuración de notificaciones (toggles) y seguridad de cuenta.

SECCIÓN 4

Justificación de Decisiones de Diseño

1. ¿Por qué elegiste ese estilo visual?

El estilo visual de AulaMatch responde al contexto operativo de sus usuarios: personal administrativo universitario que trabaja con información densa durante jornadas extensas. Se adoptó un diseño flat sin gradientes ni sombras profundas porque el protagonismo visual debía estar en los datos de la tabla, no en la decoración de la interfaz. La paleta fue construida con propósito semántico: el sidebar en #1E2A45 establece un ancla visual estable; el fondo #F7F8FC reduce la fatiga ocular; el primario #2563EB queda reservado para acciones afirmativas. Se descartó glassmorphism y tarjetas flotantes porque añaden ruido visual sin aportar jerarquía funcional. El resultado es una interfaz que comunica seriedad institucional sin caer en el aspecto obsoleto de los sistemas administrativos tradicionales.

2. ¿Qué buscaste destacar con la distribución de los elementos?

La distribución busca que el coordinador entienda el estado global de las asignaciones en el menor número de pasos visuales posibles. La tabla densa con once columnas ocupa el centro porque es el artefacto principal: cada fila es una comisión con contexto completo visible sin abrir un detalle. Los filtros superiores permiten el flujo: filtrar primero, leer después. La columna "Estado" al extremo derecho funciona como semáforo de lectura: el ojo recorre la fila y termina en la señal de estado. El sidebar fijo garantiza que la navegación entre secciones no desplace el área de trabajo.

3. ¿Qué ventajas ofrece el diseño para la experiencia del usuario?

La ventaja central es la reducción de carga cognitiva en alta densidad informativa. Los badges con color semántico —verde "Asignada", amarillo "Pendiente", rojo "Conflicto"— permiten identificar situaciones críticas sin leer cada celda. Los chips de Carrera resuelven la multiplicidad M:N entre Materia y Carrera sin romper la estructura de fila única. El Panel de Conflictos como pantalla independiente evita que el flujo de resolución interrumpa la vista de asignaciones. El botón "Asignar automáticamente" ubica la función más demandante como acción destacada, no enterrada en un menú secundario.

4. ¿Qué mejoras detectaste durante el proceso y cómo las resolviste?

El proceso incluyó dos iteraciones concretas. Primera: abandono del calendario como pantalla principal. La propuesta inicial usaba una vista tipo calendario para las asignaciones, pero al poblarla con datos reales la pantalla se volvía ilegible —bloques comprimidos, filtros cruzados imposibles, navegación semanal que fragmentaba la visión global. La migración a tabla densa con filtros superiores resolvió estos problemas. Segunda: incorporación de chips de Carrera. Al incluir la entidad Carrera con relación M:N hacia Materia, la columna no podía ser un valor único: se optó por pills visuales compactas en #DBEAFE que contienen esa multiplicidad sin requerir celda expansible ni tooltip adicional.

SECCIÓN 5

Diagramas UML — Modelo de Dominio

A continuación se presentan los artefactos UML que sustentan el diseño de AulaMatch: el diagrama de clases y el diagrama de casos de uso. Ambos fueron elaborados de forma iterativa junto con el diseño de la interfaz, asegurando coherencia entre el modelo de dominio y las decisiones de UX.

5.1 — Diagrama de Clases

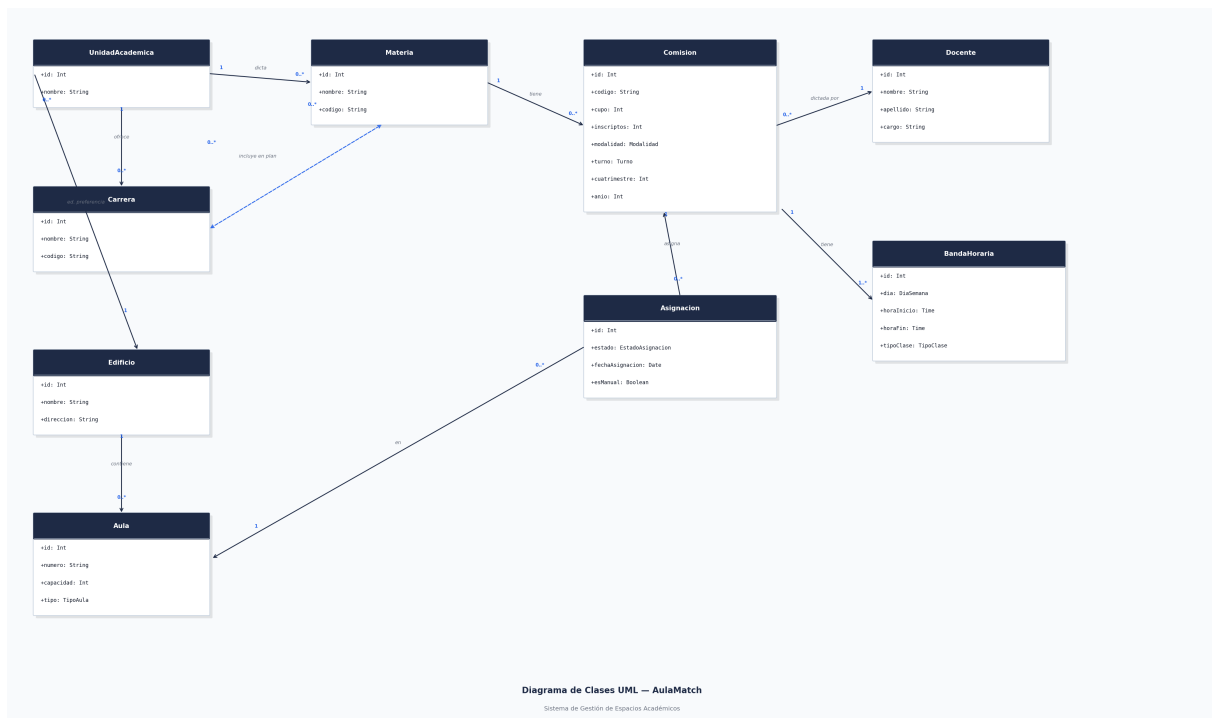


Figura 5 — Diagrama de Clases UML de AulaMatch. Muestra las 9 entidades del dominio y sus relaciones, incluyendo la entidad Carrera con su relación M:N hacia Materia incorporada en la segunda iteración del modelo.

Decisiones de modelado

1. Asignacion como entidad independiente:

Centraliza el vínculo entre Comision y Aula, evitando una relación M:N directa y permitiendo registrar metadatos propios (estado, fecha, modo de asignación).

2. BandaHoraria como entidad separada:

Una comisión puede cursar más de un día por semana; modelarla como entidad propia permite representar la cardinalidad 1..* sin campos repetidos en Comision.

3. Edificio como entidad central:

Actúa como eje físico del modelo, conectando Aula con UnidadAcademica sin crear dependencias directas entre ellas.

4. Tipos de dato semánticos (TipoAula, Modalidad, Turno, DiaSemana, EstadoAsignacion):

Se declaran como tipos nombrados en lugar de String para comunicar que serán enumeraciones controladas en la base de datos.

5. Carrera con atributos id, nombre y codigo únicamente:

En el alcance de AulaMatch, Carrera es una entidad de referencia. Su código actúa como clave natural alineada con SIU Guaraní, facilitando la integración.

6. Relación Carrera ↔ Materia como M:N directa, sin entidad intermedia:

La pertenencia de una materia a una carrera no tiene atributos propios en el alcance actual. La tabla de unión carrera_materia se generará al mapear el modelo a la base relacional. Si en el futuro se requieren atributos (tipo, añoEnPlan), esa tabla se eleva a entidad nombrada sin romper el modelo.

7. Coexistencia de UA → Materia y UA → Carrera → Materia:

La relación directa expresa responsabilidad académica (quién dicta la materia); la ruta indirecta expresa pertenencia curricular (en qué plan figura). Son semánticas distintas y no redundantes.

5.2 — Diagrama de Casos de Uso

El diagrama de casos de uso identifica los dos actores humanos del sistema (Coordinador de Espacios y Administrativo de UA) y el actor sistema (procesos automáticos), organizados en cuatro módulos funcionales:

- **Gestión de Espacios Físicos:** Registrar edificio, aula y sus características. Responsabilidad exclusiva del Coordinador de Espacios.
- **Gestión de Comisiones:** Consultar oferta, registrar comisión, asignar aula (automática o manualmente), consultar disponibilidad y gestionar excepciones.
- **Detección y Alertas:** Detectar conflictos de horario y exceso de cupo (actor Sistema), disparando siempre la notificación al usuario mediante <<include>>.
- **Consultas Transversales:** Consultar estado de asignaciones y exportar reportes, accesibles para ambos actores humanos.

Decisiones de modelado del diagrama de casos de uso

UC «include» Consultar aulas disponibles:

Tanto la asignación automática como la manual siempre requieren consultar disponibilidad; extraerlo evita duplicación y deja en claro que es una operación central del sistema.

UC «extend» Gestionar excepción:

Es un flujo alternativo que ocurre solo ante situaciones imprevistas, no un paso obligatorio, lo que justifica la relación de extensión sobre Asignar aula manualmente.

Notificar conflicto incluido por Detectar conflicto y Detectar exceso de cupo:

Toda detección automática siempre dispara una notificación; el «include» refleja que la notificación no es opcional ni separada del proceso de detección.

Filtrado por Carrera en UC4, UC13 y UC14:

La incorporación de Carrera al modelo habilita filtrar comisiones por plan de estudios y agrupar reportes por carrera, sin agregar casos de uso nuevos.

SECCIÓN 6

Diagrama de Arquitectura Lógica

El diagrama siguiente presenta la arquitectura lógica de AulaMatch organizada en tres capas bien diferenciadas: Presentación, Lógica de Negocio y Persistencia. Las flechas muestran el flujo de datos entre pantallas, módulos y tablas relacionales.

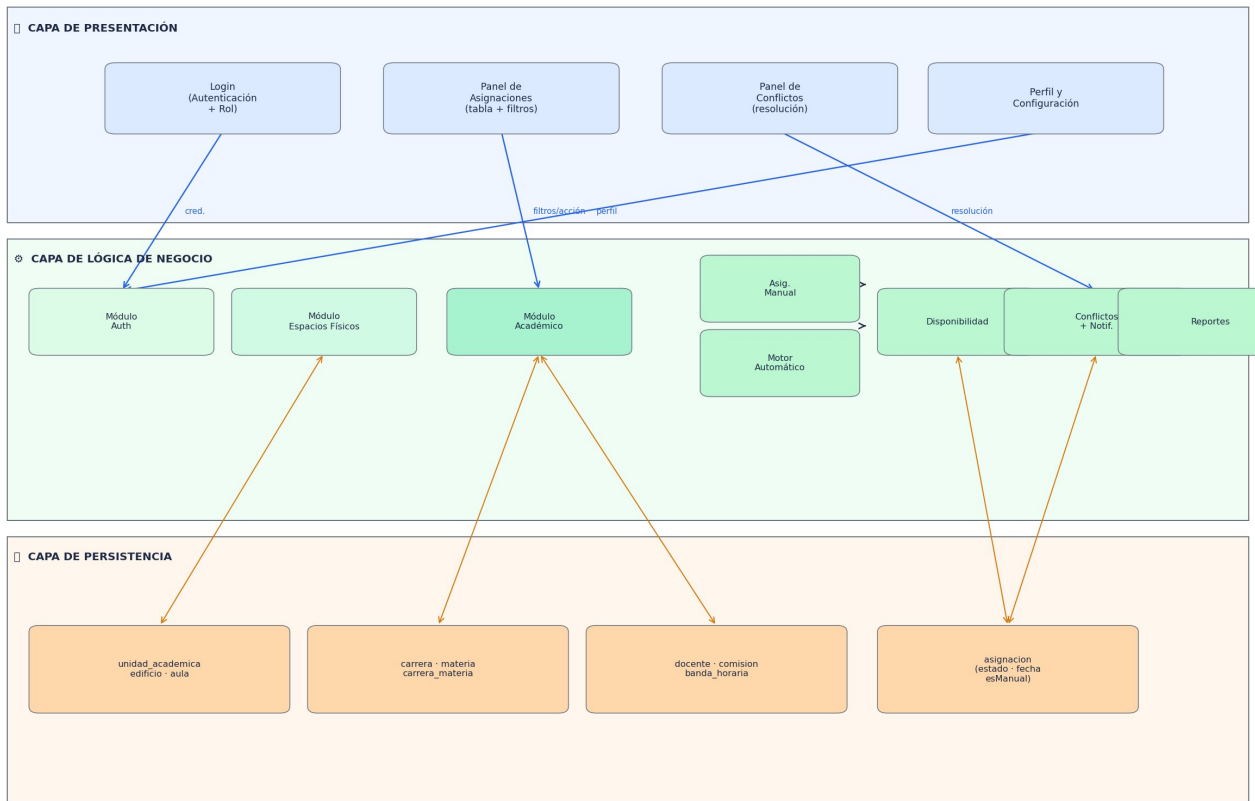


Figura 6 — Diagrama de Arquitectura Lógica de AulaMatch. Tres capas: Presentación (azul), Lógica de Negocio (verde) y Persistencia (naranja).

Explicación por capa

Capa de Presentación — Interfaz web responsive ejecutada en el navegador. Comprende cuatro pantallas funcionales: el *Login* autentica al usuario y establece su rol (Coordinador de Espacios o Administrativo de UA); el *Panel de Asignaciones* es la vista central con tabla densa, filtros y badges de estado semánticos; el *Panel de Conflictos* muestra alertas activas y ofrece resolución asistida; y *Perfil / Configuración* gestiona datos institucionales y preferencias de notificación.

Capa de Lógica de Negocio — Monolito modular con seis módulos de responsabilidad acotada: *Autenticación* controla sesiones y permisos mediante JWT; *Espacios Físicos* gestiona el CRUD de Edificios y Aulas; *Académico* nuclea UA, Carreras, Materias, Docentes y Comisiones, incluyendo la integración propuesta con SIU Guaraní; *Asignación* contiene el motor automático y el flujo manual asistido, ambos apoyados en la consulta de disponibilidad; *Conflictos* corre consultas cruzadas y dispara notificaciones; *Reportes* provee

exportación transversal.

Capa de Persistencia — Base de datos relacional (PostgreSQL) que implementa las 9 entidades del modelo de dominio: *UnidadAcademica*, *Edificio*, *Aula*, *Carrera*, *Materia*, *Docente*, *Comision*, *BandaHoraria* y *Asignacion*, con la tabla de unión *carrera_materia* para la relación M:N. La integridad referencial y las consultas de disponibilidad y conflicto son el fundamento técnico del producto.

SECCIÓN 7

Diagramas UML Adicionales

7.1 — Diagrama de Secuencia: Resolución de Conflicto de Superposición Horaria

El diagrama cubre el flujo completo de resolución de un conflicto de superposición horaria en AulaMatch: desde la detección automática por el **Servicio de Conflictos** (que marca la *Asignacion* como "CONFLICTO"), pasando por la consulta de aulas compatibles filtradas por cupo, tipo y disponibilidad horaria, hasta la confirmación manual del **Coordinador de Espacios**, que actualiza la *Asignacion* a estado "ASIGNADA" con *esManual* = true.

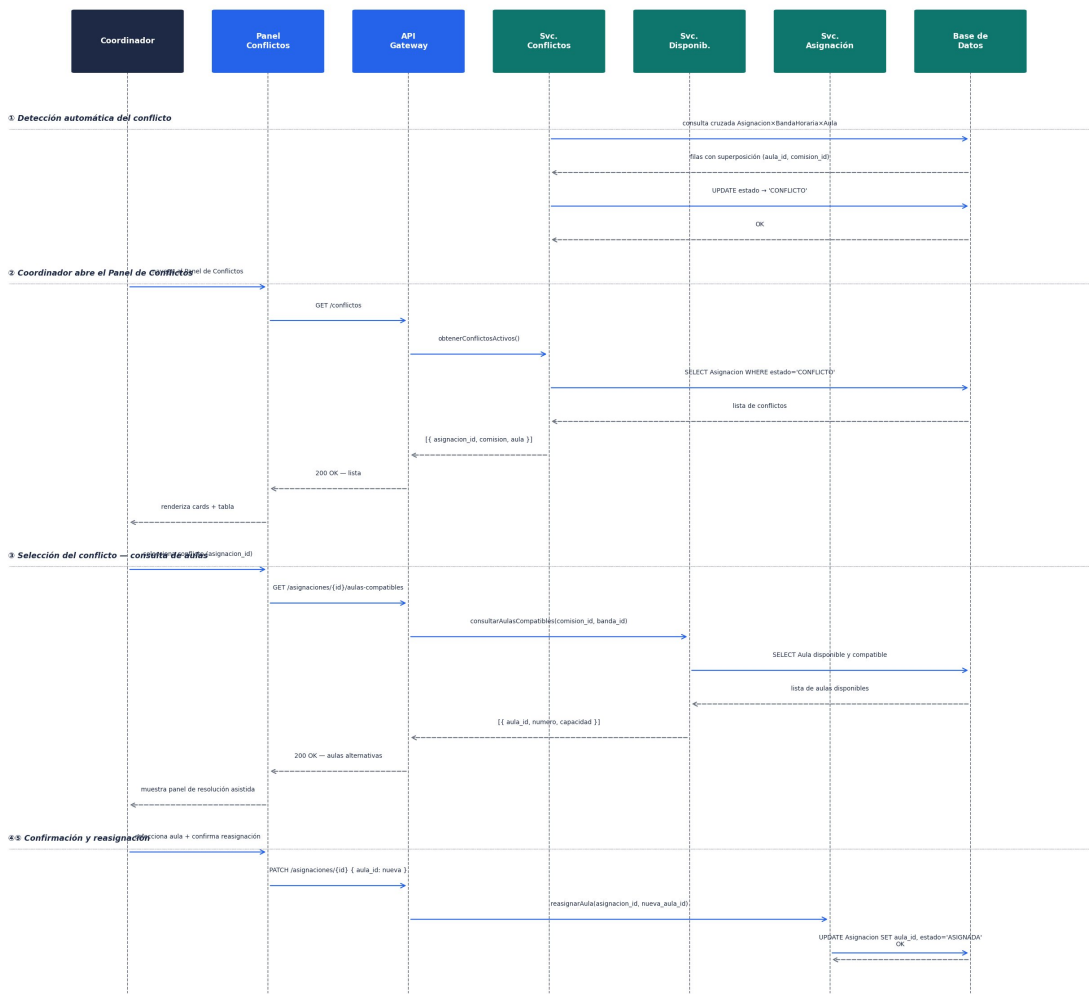


Figura 7.1 — Diagrama de Secuencia UML: Resolución de conflicto de superposición horaria. Participantes: Coordinador, Panel de Conflictos, API Gateway, Svc. Conflictos, Svc. Disponibilidad, Svc. Asignación y Base de Datos.

Pasos del flujo

1. Detección automática — El Servicio de Conflictos ejecuta una consulta cruzada sobre Asignacion x BandaHoraria x Aula. Cuando detecta superposición, actualiza el estado de la Asignacion a "CONFLICTO".

2. Apertura del panel — El Coordinador navega al Panel de Conflictos. El frontend realiza GET /conflictos y renderiza las cards de métricas y la tabla de asignaciones en conflicto.

3. Selección y consulta — Al seleccionar un conflicto, el frontend solicita GET /asignaciones/{id}/aulas-compatibles. El Servicio de Disponibilidad filtra aulas por capacidad, tipo de clase y franja horaria libre.

4. Confirmación — El Coordinador elige un aula alternativa y confirma. El frontend envía PATCH /asignaciones/{id} con el nuevo aula_id.

5. Reasignación — El Servicio de Asignación actualiza la Asignacion: aula_id = nueva, estado = "ASIGNADA", esManual = true. El badge cambia a verde en la interfaz.

7.2 — Diagrama de Actividades: Detección de Conflictos en Asignaciones

El proceso se dispara cada vez que una *Asignacion* es creada o modificada. Realiza **dos verificaciones independientes y secuenciales**. Cada una dispara una notificación obligatoria e inmediata al detectar un problema. Si ninguna verificación detecta conflicto, la *Asignacion* conserva el estado "*ASIGNADA*".

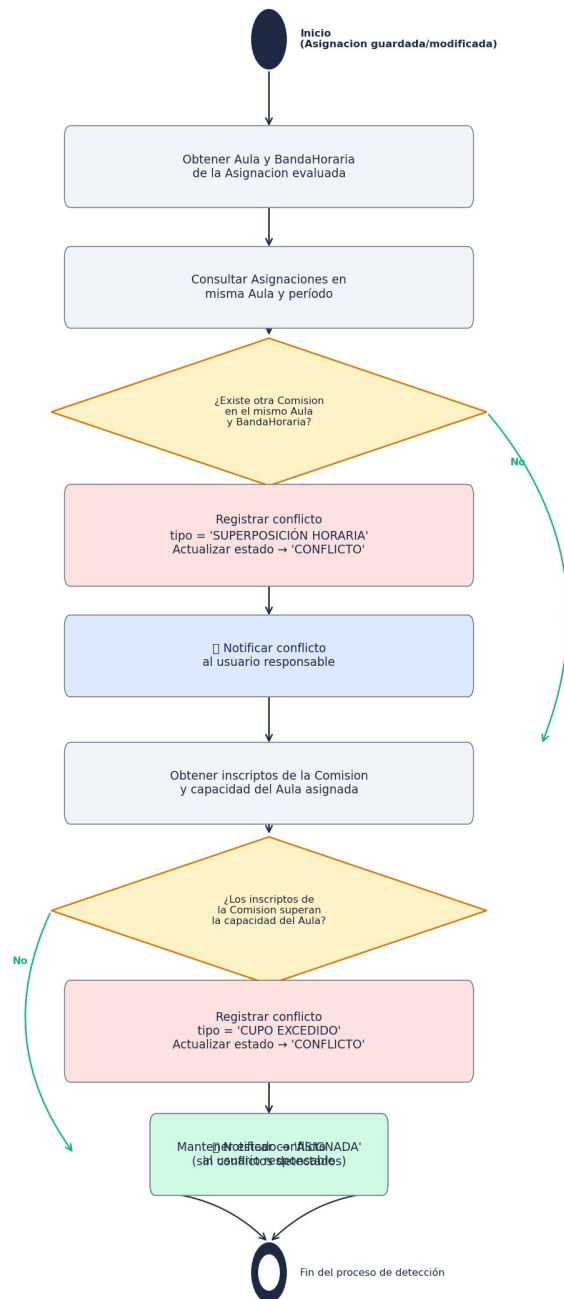


Figura 7.2 — Diagrama de Actividades UML: Detección de conflictos en asignaciones. Verificación 1: superposición horaria. Verificación 2: cupo excedido. Ambas disparan notificación obligatoria.

Lógica del proceso

Verificación 1 — Superposición horaria: Consulta si existe otra Comision asignada al mismo Aula con una BandaHoraria solapada. Si la hay, registra el conflicto (tipo = "Superposición horaria") y notifica al usuario antes de continuar.

Verificación 2 — Cupo excedido: Compara el campo *inscriptos* de la Comision contra la *capacidad* del Aula. Si el cupo es insuficiente, registra el conflicto (tipo = "Cupo excedido") y notifica.

SECCIÓN 8

Estructuras de Datos

Nota: Las estructuras que siguen son una propuesta derivada del diagrama de clases UML y las decisiones de modelado documentadas. Son coherentes con la arquitectura de monolito modular y base de datos relacional definida en el proyecto.

8.1 — Enumeraciones controladas

Enumeración	Valores permitidos
TipoAula	AULA · LABORATORIO · AUDITORIO · SALA_VIDEOCONFERENCIA
Modalidad	PRESENCIAL · VIRTUAL · HIBRIDA
Turno	MANANA · TARDE · NOCHE
DiaSemana	LUNES · MARTES · MIERCOLES · JUEVES · VIERNES · SABADO
EstadoAsignacion	PENDIENTE · ASIGNADA · CONFLICTO
TipoClase	TEORICA · PRACTICA · TEORICO_PRACTICA

8.2 — Esquema relacional

unidad_academica

Columna	Tipo	Restricciones
id	SERIAL	PK
nombre	VARCHAR(120)	NOT NULL, UNIQUE
edificio_preferencia_id	INTEGER	FK → edificio(id), NULLABLE

edificio

Columna	Tipo	Restricciones
id	SERIAL	PK
nombre	VARCHAR(100)	NOT NULL
direccion	VARCHAR(200)	NOT NULL

aula

Columna	Tipo	Restricciones
id	SERIAL	PK
numero	VARCHAR(20)	NOT NULL
capacidad	INTEGER	NOT NULL, CHECK > 0
tipo	TipoAula	NOT NULL (enumeración)

Columna	Tipo	Restricciones
edificio_id	INTEGER	FK → edificio(id), NOT NULL

Índices: idx_aula_edificio (edificio_id) · idx_aula_tipo_capacidad (tipo, capacidad)

carrera

Columna	Tipo	Restricciones
id	SERIAL	PK
nombre	VARCHAR(120)	NOT NULL
codigo	VARCHAR(20)	NOT NULL, UNIQUE
unidad_academica_id	INTEGER	FK → unidad_academica(id), NOT NULL

materia

Columna	Tipo	Restricciones
id	SERIAL	PK
nombre	VARCHAR(120)	NOT NULL
codigo	VARCHAR(20)	NOT NULL, UNIQUE
unidad_academica_id	INTEGER	FK → unidad_academica(id), NOT NULL

carrera_materia [tabla de unión M:N]

Columna	Tipo	Restricciones
carrera_id	INTEGER	FK → carrera(id), NOT NULL
materia_id	INTEGER	FK → materia(id), NOT NULL

PK compuesta: (carrera_id, materia_id) · Índice: idx_carrera_materia_materia (materia_id)

docente

Columna	Tipo	Restricciones
id	SERIAL	PK
nombre	VARCHAR(80)	NOT NULL
apellido	VARCHAR(80)	NOT NULL
cargo	VARCHAR(100)	NULLABLE

comision

Columna	Tipo	Restricciones
id	SERIAL	PK

Columna	Tipo	Restricciones
codigo	VARCHAR(30)	NOT NULL, UNIQUE
cupo	INTEGER	NOT NULL, CHECK > 0
inscriptos	INTEGER	NOT NULL, DEFAULT 0
modalidad	Modalidad	NOT NULL (enumeración)
turno	Turno	NOT NULL (enumeración)
cuatrimestre	SMALLINT	NOT NULL, CHECK IN (1, 2)
anio	SMALLINT	NOT NULL
materia_id	INTEGER	FK → materia(id), NOT NULL
docente_id	INTEGER	FK → docente(id), NOT NULL

Índices: *idx_comision_materia* (materia_id) · *idx_comision_periodo* (anio, cuatrimestre)

banda_horaria

Columna	Tipo	Restricciones
id	SERIAL	PK
dia	DiaSemana	NOT NULL (enumeración)
hora_inicio	TIME	NOT NULL
hora_fin	TIME	NOT NULL, CHECK > hora_inicio
tipo_clase	TipoClase	NOT NULL (enumeración)
comision_id	INTEGER	FK → comision(id), NOT NULL

Índice: *idx_banda_comision* (comision_id)

asignacion

Columna	Tipo	Restricciones
id	SERIAL	PK
estado	EstadoAsignacion	NOT NULL, DEFAULT "PENDIENTE"
fecha_asignacion	TIMESTAMP	NOT NULL, DEFAULT NOW()
es_manual	BOOLEAN	NOT NULL, DEFAULT FALSE
comision_id	INTEGER	FK → comision(id), NOT NULL
aula_id	INTEGER	FK → aula(id), NOT NULL

Índices: *idx_asignacion_comision* (comision_id) · *idx_asignacion_aula_estado* (aula_id, estado) · *idx_asignacion_estado* (estado)

8.3 — Ejemplo de respuesta de API

El siguiente objeto JSON ilustra la respuesta completa de `GET /api/asignaciones/COM-2025-001`, tal como alimenta una fila del Panel de Asignaciones del prototipo funcional.

```
{
  "id": 1,
  "estado": "ASIGNADA",
  "fechaAsignacion": "2025-03-10T09:42:00Z",
  "esManual": false,
  "comision": {
    "id": 101,
    "codigo": "COM-2025-001",
    "cupo": 40,
    "inscriptos": 37,
    "modalidad": "PRESENCIAL",
    "turno": "MANANA",
    "cuatrimestre": 1,
    "anio": 2025,
    "materia": {
      "id": 12,
      "nombre": "Bases de Datos I",
      "codigo": "BD-101",
      "unidadAcademica": { "id": 3, "nombre": "Facultad de Ingeniería" },
      "carreras": [
        { "id": 5, "nombre": "Ingeniería en Sistemas", "codigo": "IS" },
        { "id": 7, "nombre": "Licenciatura en Informática", "codigo": "LI" }
      ]
    },
  },
  "docente": {
    "id": 22, "nombre": "Lucía", "apellido": "Fernández",
    "cargo": "Profesora Adjunta"
  },
  "bandasHorarias": [
    { "id": 88, "dia": "LUNES", "horaInicio": "08:00",
      "horaFin": "10:00", "tipoClase": "TEORICA" },
    { "id": 89, "dia": "MIERCOLES", "horaInicio": "08:00",
      "horaFin": "10:00", "tipoClase": "PRACTICA" }
  ],
  "aula": {
    "id": 34, "numero": "A-204", "capacidad": 45, "tipo": "AULA",
    "edificio": {
      "id": 2, "nombre": "Edificio Aulas Norte",
      "direccion": "Av. General Paz 5445, San Martín"
    }
  }
}
```

Campo	Decisión de diseño
carreras (array en materia)	Resuelve la relación M:N Carrera ↔ Materia; alimenta los chips/pills de la columna "Carreras" del prototipo
bandasHorarias (array)	Refleja cardinalidad 1..* de BandaHoraria sobre Comision; una misma comisión puede cursar más de un día
esManual: false	Indica que la asignación fue generada por el motor automático, no por intervención del coordinador
estado: "ASIGNADA"	Badge verde del prototipo. Otros valores: "PENDIENTE" (amarillo) · "CONFLICTO" (rojo)
inscriptos: 37 / capacidad: 45	Cupo no excedido. Si inscriptos > capacidad, el Servicio de Conflictos marcaría estado: "CONFLICTO"

SECCIÓN 9

Guía de Despliegue en Servidor

Nota: Esta guía es una propuesta completa coherente con la arquitectura lógica documentada (monolito modular, base de datos relacional, aplicación web). Los archivos de configuración reales se encuentran en la carpeta deploy/ del proyecto.

Requisitos previos del servidor

Requisito	Especificación mínima recomendada
Sistema operativo	Ubuntu 22.04 LTS (o Debian 12)
CPU	2 vCPU
RAM	4 GB
Disco	40 GB SSD
Docker	24.x o superior
Docker Compose	v2.x (incluido en Docker Engine)
Dominio	Nombre de dominio apuntando al servidor (ej: aulamatch.universidad.edu.ar)
Puertos	80 y 443 abiertos en el firewall del servidor

Paso 1 — Instalar Docker en el servidor

Conectarse al servidor por SSH y ejecutar los siguientes comandos para agregar el repositorio oficial de Docker e instalar Docker Engine y Docker Compose:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
sudo usermod -aG docker $USER && newgrp docker
```

Paso 2 — Clonar el proyecto y crear la estructura de directorios

Crear el directorio raíz del proyecto y clonar (o subir) el código fuente:

```
mkdir -p /opt/aulamatch && cd /opt/aulamatch
# git clone https://github.com/tu-org/aulamatch.git .
# Estructura esperada:
# /opt/aulamatch/
#   ├── deploy/
#   │   ├── docker-compose.yml
#   │   ├── nginx.conf
#   │   └── init-db/01_schema.sql
#   ├── backend/      ← API REST (Node.js + Express)
#   ├── frontend/     ← React compilado
#   └── .env           ← NO subir a Git
```

Paso 3 — Configurar variables de entorno

Crear el archivo `.env` en `/opt/aulamatch/` a partir del ejemplo incluido. Reemplazar todos los valores de placeholder antes de continuar:

```
cp /opt/aulamatch/deploy/.env.example /opt/aulamatch/.env

# Contenido de .env:
POSTGRES_DB=aulamatch
POSTGRES_USER=aulamatch_user
POSTGRES_PASSWORD=CambiarPorContraseñaSegura123!
DATABASE_URL=postgresql://aulamatch_user:...@db:5432/aulamatch
JWT_SECRET=CambiarPorSecretoAleatorio64Chars
JWT_EXPIRES_IN=8h
NODE_ENV=production
PORT=3001
VITE_API_BASE_URL=https://aulamatch.universidad.edu.ar/api
DOMAIN=aulamatch.universidad.edu.ar
CERTBOT_EMAIL=admin@universidad.edu.ar
```

Paso 4 — Revisar la estructura de servicios Docker Compose

El archivo `deploy/docker-compose.yml` define cuatro servicios con sus responsabilidades:

Paso 5 — Ejecutar migraciones de base de datos

El esquema inicial se aplica automáticamente al primer inicio mediante `deploy/init-db/01_schema.sql`. Para migraciones posteriores:

```
docker compose -f /opt/aulamatch/deploy/docker-compose.yml exec db \
  psql -U aulamatch_user -d aulamatch
# Verificar tablas:
\dt

# Si el backend usa Prisma:
docker compose -f deploy/docker-compose.yml exec backend \
  npx prisma migrate deploy
```

Paso 6 — Obtener certificado HTTPS con Let's Encrypt

Antes de iniciar Nginx con HTTPS, obtener el certificado SSL gratuito:

```
sudo apt install -y certbot
sudo certbot certonly --standalone \
  -d aulamatch.universidad.edu.ar \
  --email admin@universidad.edu.ar \
  --agree-tos --non-interactive
# Certificado en: /etc/letsencrypt/live/aulamatch.universidad.edu.ar/
```

Paso 7 — Iniciar todos los servicios

Construir las imágenes y levantar todos los contenedores en segundo plano:

```
cd /opt/aulamatch
docker compose -f deploy/docker-compose.yml --env-file .env up -d --build
docker compose -f deploy/docker-compose.yml ps

# Salida esperada:
# aulamatch-db-1      running  5432/tcp
# aulamatch-backend-1 running  3001/tcp
# aulamatch-frontend-1 running  80/tcp
# aulamatch-nginx-1   running  0.0.0.0:80->80, 0.0.0.0:443->443
```

Paso 8 — Verificar el despliegue

Verificar logs y endpoints principales:

```
# Logs de servicios
docker compose -f deploy/docker-compose.yml logs backend
docker compose -f deploy/docker-compose.yml logs nginx

# Health check de la API (debe responder 200 OK)
curl -s https://aulamatch.universidad.edu.ar/api/health
# Respuesta esperada: {"status":"ok","db":"connected"}
```

Paso 9 — Operaciones de mantenimiento

Comandos esenciales para el ciclo de vida de la aplicación:

```
# Reiniciar un servicio
docker compose -f deploy/docker-compose.yml restart backend

# Actualizar la aplicación
git pull origin main
docker compose -f deploy/docker-compose.yml up -d --build

# Backup de la base de datos
docker compose -f deploy/docker-compose.yml exec db pg_dump \
-U aulamatch_user aulamatch > aulamatch_$(date +%Y%m%d).sql

# Detener (sin borrar datos)
docker compose -f deploy/docker-compose.yml down
```

Servicio	Imagen base	Responsabilidad
db	postgres:16-alpine	Base de datos relacional PostgreSQL
backend	Build local desde ./backend	API REST — monolito modular Node.js + Express
frontend	Build local desde ./frontend	Aplicación React compilada (Nginx interno)
nginx	nginx:alpine	Proxy reverso, HTTPS (Let's Encrypt), enrutamiento /api vs /

Arquitectura de red del despliegue

Nginx enruta `/api/*` al backend y todo lo demás al frontend estático, dentro de la red interna Docker `aulamatch_net`. El tráfico externo ingresa únicamente por los puertos 80 (redirige a 443) y 443 (HTTPS con certificado Let's Encrypt).

SECCIÓN 10

Backend: Lógica del Servidor

*Nota: El proyecto no contiene código fuente de backend implementado. Todo lo que sigue es una propuesta técnica derivada de las funcionalidades documentadas en el prototipo, el modelo de datos y la arquitectura lógica. El stack propuesto es **Node.js + Express** sobre la arquitectura de monolito modular definida.*

10.1 — Estructura de módulos del backend

```
backend/
├── src/
│   ├── modules/
│   │   ├── auth/           ← Autenticación y autorización por rol
│   │   ├── asignaciones/   ← Motor automático + asignación manual
│   │   ├── conflictos/     ← Detección de conflictos y notificaciones
│   │   ├── aulas/         ← CRUD de Edificio y Aula
│   │   ├── academico/     ← UA, Carrera, Materia, Docente, Comisión
│   │   ├── reportes/      ← Exportación y consultas transversales
│   │   └── middlewares/
│   │       ├── authenticate.js ← Validación de JWT
│   │       ├── authorize.js   ← Validación de rol por endpoint
│   │       └── app.js
└──
```

10.2 — Endpoints principales por módulo

Módulo: Autenticación

Método	Endpoint	Descripción	Roles
POST	/api/auth/login	Valida credenciales y devuelve JWT	Público
POST	/api/auth/logout	Invalida la sesión activa	Autenticado
GET	/api/auth/me	Devuelve datos del usuario autenticado	Autenticado

Módulo: Asignaciones

Método	Endpoint	Descripción	Roles
GET	/api/asignaciones	Lista asignaciones con filtros (UA, carrera, edificio, turno, modalidad, estado)	Coordinador, Administrativo
POST	/api/asignaciones/automatica	Ejecuta el motor de asignación automática para el período indicado	Coordinador, Administrativo
POST	/api/asignaciones	Crea una asignación manual para una comisión específica	Coordinador, Administrativo
PATCH	/api/asignaciones/:id	Reasigna o actualiza una asignación (incluye resolución de conflictos)	Coordinador, Administrativo
GET	/api/asignaciones/:id/aulas-compatibles	Lista aulas disponibles y compatibles para una comisión dada	Coordinador, Administrativo
DELETE	/api/asignaciones/:id	Elimina una asignación (libera el aula)	Coordinador

Módulo: Conflictos

Método	Endpoint	Descripción	Roles
GET	/api/conflictos	Lista asignaciones con estado = "CONFLICTO"	Coordinador, Administrativo
GET	/api/conflictos/metricas	Devuelve contadores para las cards del Panel de Conflictos	Coordinador, Administrativo
POST	/api/conflictos/detectar	Fuerza la detección manual de conflictos para el período activo	Coordinador

Módulo: Aulas

Método	Endpoint	Descripción	Roles
GET	/api/edificios	Lista todos los edificios	Coordinador, Administrativo
POST	/api/edificios	Registra un nuevo edificio	Coordinador
GET	/api/edificios/:id/aulas	Lista las aulas de un edificio	Coordinador, Administrativo
POST	/api/aulas	Registra un aula con sus características	Coordinador
PATCH	/api/aulas/:id	Actualiza datos o tipo de un aula	Coordinador

Módulo: Académico

Método	Endpoint	Descripción	Roles
GET	/api/unidades-academicas	Lista todas las UAs	Coordinador, Administrativo
GET	/api/carreras	Lista carreras (filtrable por UA)	Coordinador, Administrativo
GET	/api/materias	Lista materias (filtrable por UA o carrera)	Coordinador, Administrativo
GET	/api/comisiones	Lista comisiones del período activo con filtros	Coordinador, Administrativo
POST	/api/comisiones	Registra una comisión nueva (gestión de excepciones en tiempo real)	Administrativo
GET	/api/docentes	Lista docentes	Coordinador, Administrativo

Módulo: Reportes

Método	Endpoint	Descripción	Roles
GET	/api/reportes/asignaciones	Exporta el estado de asignaciones del período (JSON / CSV)	Coordinador, Administrativo
GET	/api/reportes/disponibilidad	Muestra ocupación de aulas por edificio y franja horaria	Coordinador

Método	Endpoint	Descripción	Roles
GET	/api/health	Estado del servidor y conexión a base de datos	Público

10.3 — Algoritmo de asignación automática

El motor de asignación automática se ejecuta al recibir *POST /api/asignaciones/automatica* con el período lectivo como parámetro. Su objetivo es proponer la asignación óptima para cada comisión sin aula, respetando cuatro restricciones simultáneas: **cupo, tipo de aula, franja horaria disponible y edificio preferido de la UA.**

```

FUNCIÓN asignarAutomaticamente(anio, cuatrimestre):

    comisionesSinAula ← obtenerComisiones(
        anio=anio, cuatrimestre=cuatrimestre, estado=SIN_ASIGNACION
    )
    // Ordenar por demanda: primero las de mayor cupo (más difíciles de ubicar)
    comisionesSinAula ← ordenarPorCupoDescendente(comisionesSinAula)

    asignacionesPropuestas ← []
    asignacionesFallidas ← []

    PARA CADA comision EN comisionesSinAula:
        bandasHorarias ← comision.bandasHorarias
        cupoRequerido ← comision.inscriptos
        tipoClaseReq ← inferirTipoAula(comision.modalidad, bandasHorarias.tipoClase)
        edificioPref ← comision.materia.unidadAcademica.edificioPreferencia

        // 1. Buscar aulas candidatas por tipo y cupo
        aulasCandidatas ← buscarAulas(tipo >= tipoClaseReq, capacidad >= cupoRequerido)

        // 2. Priorizar edificio preferido de la UA
        aulasPreferidas ← filtrarPorEdificio(aulasCandidatas, edificioPref)
        aulasAlternativas ← aulasCandidatas - aulasPreferidas
        aulasCandidatas ← aulasPreferidas + aulasAlternativas

        // 3. Filtrar por disponibilidad horaria (sin superposición)
        aulasDisponibles ← []
        PARA CADA aula EN aulasCandidatas:
            SI NO existeSuperposicion(aula, bandasHorarias):
                aulasDisponibles.agregar(aula)

        // 4. Seleccionar la mejor aula disponible (menor capacidad sobrante)
        SI aulasDisponibles NO está vacío:
            aulaElegida ← aulasDisponibles[0]
            asignacionesPropuestas.agregar(
                crearAsignacion(comision.id, aulaElegida.id, estado="ASIGNADA",
                    esManual=false, fechaAsignacion=AHORA())
            )
        SINO:
            asignacionesFallidas.agregar({comision, motivo="SIN_AULA_DISPONIBLE"})

    guardarEnBD(asignacionesPropuestas)
    RETORNAR { asignadas, pendientes, resumen: {total, asignadas, fallidas} }

FIN FUNCIÓN

```

10.4 — Autenticación y autorización por rol (JWT)

El sistema utiliza **JSON Web Tokens (JWT)** para gestionar sesiones sin estado (*stateless*), coherente con la arquitectura de API REST. Los roles definidos son:

Rol	Descripción	Alcance
COORDINADOR	Coordinador de Espacios	Acceso total: espacios físicos, asignaciones, reportes globales
ADMINISTRATIVO	Administrativo de UA	Acceso operativo: comisiones y asignaciones de su propia UA

Estructura del token JWT generado por *POST /api/auth/login*:

```
{
  "header": { "alg": "HS256", "typ": "JWT" },
  "payload": {
    "sub": 42,
    "nombre": "Lucía Fernández",
    "rol": "COORDINADOR",
    "unidadAcademicaId": 3,
    "iat": 1720000000,
    "exp": 1720028800    // 8 horas de vigencia
  }
}
```

El **middleware authenticate.js** verifica la firma y vigencia del token en cada request protegido. El **middleware authorize.js** valida el rol requerido por endpoint. El campo *unidadAcademicaId* embebido en el JWT permite que los controladores apliquen automáticamente el filtro de UA para usuarios con rol ADMINISTRATIVO, sin requerir que el cliente lo envíe explícitamente.

```
// Ejemplo de aplicación en Express (propuesta):
router.post('/edificios',
  authenticate,           // ← verifica JWT
  authorize(['COORDINADOR']), // ← solo coordinadores
  edificiosController.crear
);

router.get('/asignaciones',
  authenticate,
  authorize(['COORDINADOR', 'ADMINISTRATIVO']),
  asignacionesController.listar // ← aplica scope de UA si es ADMINISTRATIVO
);
```

10.5 — Detección de conflictos y notificaciones

La detección de conflictos se ejecuta en **tres momentos distintos**: (1) al crear o modificar una Asignacion (trigger síncrono), (2) al finalizar la asignación automática masiva, y (3) por petición manual del coordinador vía *POST /api/conflictos/detectar*.

Verificación 1 — Superposición horaria:

```
PARA CADA banda EN comision.bandasHorarias:
  SI EXISTE otra Asignacion WHERE
    aula_id = asignacion.aula_id
    AND estado != "CONFLICTO"
    AND banda_horaria.dia = banda.dia
    AND NOT (banda_horaria.hora_fin <= banda.hora_inicio
              OR banda_horaria.hora_inicio >= banda.hora_fin):
    marcarConflicto(asignacion, tipo="SUPERPOSICION_HORARIA")
    notificar(asignacion)  // ← obligatorio e inmediato
```

Verificación 2 — Cupo excedido:

```
SI comision.inscriptos > aula.capacidad:
  marcarConflicto(asignacion, tipo="CUPO_EXCEDIDO")
  notificar(asignacion)  // ← obligatorio e inmediato
```

Cuando se detecta un conflicto, el sistema actualiza *Asignacion.estado* = "CONFLICTO" en la base de datos. En la siguiente carga del Panel de Conflictos, el badge del ícono de alerta muestra el contador actualizado desde *GET /api/conflictos/metricas*. La notificación utiliza polling activo desde el frontend en la versión inicial; WebSockets pueden incorporarse como mejora futura.

10.6 — Trazabilidad funcionalidad → módulo → entidades

Funcionalidad	Módulo / Endpoint	Entidades
Asignación automática masiva	asignaciones — POST /api/asignaciones/automatica	Comision, Aula, BandaHoraria, Asignacion, Edificio
Asignación manual asistida	asignaciones — GET /api/asignaciones/:id/aulas-compatibles	Aula, BandaHoraria, Asignacion
Gestión de excepciones (nueva comisión)	academico — POST /api/comisiones + asignaciones	Comision, Materia, BandaHoraria
Detección de superposición horaria	conflictos — trigger + POST /api/conflictos/detectar	Asignacion, BandaHoraria, Aula
Detección de cupo excedido	conflictos — trigger interno	Asignacion, Comision, Aula
Notificación ante conflicto	conflictos — acción post-detección	Asignacion, (tabla notificaciones)
Panel de Conflictos — cards métricas	conflictos — GET /api/conflictos/metricas	Asignacion
Resolución de conflicto — reasignación	asignaciones — PATCH /api/asignaciones/:id	Asignacion, Aula
Filtros por UA, Carrera, Edificio, Turno	asignaciones — GET /api/asignaciones	Comision, Materia, Carrera, Aula, Edificio
Exportación de reportes	reportes — GET /api/reportes/asignaciones	Todas las entidades
Login y selección de rol	auth — POST /api/auth/login	(tabla de usuarios — infraestructura)